

# MoMo Lens - API Report

**Team:** Web Artisans

**Course:** Enterprise Web Development.

We built a REST API using Python (http.server) for MoMo SMS data from modified\_sms\_v2.xml. We parse XML to JSON, search with list and dictionary structures and protect every endpoint with HTTP Basic Authentication.

## 1. API security

Our API lets apps read and change transaction records. We lock every route so only clients with valid credentials can use it. We use HTTP Basic Authentication: the client sends username and password in the Authorization header (Base64). If the header is missing or wrong, we return 401 Unauthorized with {"error": "Authentication required"}.

We check credentials in api/auth.py. Defaults are admin / password; we can override them with API\_USERNAME and API\_PASSWORD. We tested with curl and Postman using valid auth and wrong passwords.

Security only works on a trusted network if traffic is encrypted. Basic Auth does not encrypt credentials by itself. We must use HTTPS in production, so passwords are not sent in plain text.

## 2. Endpoint documentation

Base URL: http://127.0.0.1:8000. Data file: data/transactions.json (from python -m dsa.parse\_xml).

| Method | Path               | What we do                                        |
|--------|--------------------|---------------------------------------------------|
| GET    | /transactions      | List all transactions (flat SMS JSON).            |
| GET    | /transactions/{id} | One transaction by id.                            |
| POST   | /transactions      | Create; body validated by api/schemas.py.         |
| PUT    | /transactions/{id} | Update; URL id must match transaction_id in body. |
| DELETE | /transactions/{id} | Remove transaction; 204 empty body on success.    |

**Example:** GET one record (with auth):

```
curl -u admin:password http://127.0.0.1:8000/transactions/76662021700
```

Response 200: JSON with id, transaction\_type, amount, currency, sender, receiver, timestamp, and related fields.

**Example:** POST (with auth and JSON body):

```
curl -u admin:password -X POST http://127.0.0.1:8000/transactions -H "Content-Type: application/json" --data-binary @examples/complex_transaction.json
```

Errors we return:

- 401: no auth or wrong username/password
- 404: unknown id or bad path
- 400: invalid JSON or validation failure
- 409: POST when transaction\_id already exists

Full request/response examples: `docs/api_docs.md` in our GitHub repo.

### 3. Data parsing (context)

We parse `modified_sms_v2.xml` in `dsa/parse_xml.py` into a list of Python dictionaries (transaction type, amount, sender, receiver, timestamp, fees, balance, and id). We loaded 1,683 records. The API reads the same from `data/transactions.json`.

### 4. DSA comparison results

We store transactions in a list (linear search) and a dictionary id, record (lookup by key). We compare both in `dsa/search.py` and `dsa/benchmark.py`.

We timed 20 IDs spread across the dataset (200 repeats per id). Results are saved in `docs/benchmark_results.json`:

- Dataset size: 1,683 transactions
- Average linear search: 37.54 microseconds
- Average dictionary lookup: 0.08 microseconds
- Dictionary lookup was about 448× faster on average

**Linear search** walks through the list until it finds the id, cost grows with size ( $O(n)$ ).

**Dictionary lookup** uses the id as a key, then average  $O(1)$ . So, our GET `/transactions/{id}` uses the dictionary through the shared store in `api/db.py`.

Another option at larger scale is a sorted list with binary search ( $O(\log n)$ ), or a database index. For in-memory lookup by id, a hash map (dictionary) is the simplest fast choice for us.

## **5. Basic Auth - limits and better options**

Basic Auth was enough for this assignment, but we would not rely on it alone in production:

- Username and password go on every request; Base64 is not encryption.
- Without HTTPS, anyone on the network can capture these credentials.
- There is no token expiry, so a stolen password works until we rotate it.
- One shared login cannot express different roles (read-only vs admin).

Stronger approaches we would consider next: JWT (login once, send a signed token with expiry on later calls) and OAuth2 (delegated login and scoped access tokens without sharing our password with every client).

## **6. Testing**

We tested with curl and Postman: Success on the authenticated GET, We got 401 with wrong credentials and successful POST, PUT and DELETE. Screenshots are in the screenshots/ folder. We also use scripts/test\_api.ps1 for a quick check.